

WEB SCRAPING

Professor: Horacio Larreguy

TA: Eduardo Zago, Ernesto Dávila, Jose Luis Perez

ITAM Applied Research 2,

24/01/2024

GENERAL PERSPECTIVE

INTRODUCTION TO WEB SCRAPING

WEB SCRAPING IN PYTHON

PRACTICAL CASE: SCRAPING ITAM'S
WEBPAGE

BUILDING A BOT USING SELENIUM

SCRAPING NEWSPAPER SITES

WHAT IS WEB SCRAPING?

- ▶ Web scraping is the process of extracting data from websites.
- ▶ It involves fetching the web page's HTML and then extracting the required information from it.
- ▶ **Common practical/business uses:** extracting prices from competitors, generating potential leads using social media, checking customer satisfaction.
- ▶ **Common research uses:** coverage of major events by news media, social media, automatizing government data extraction.

COMMON PIPELINE

- ▶ **Setup:** understand what we want to do and find sources to help us do it
- ▶ **Acquisition:** read in the raw data from online and format it
- ▶ **Processing:** do text analysis, understand patterns, define your own data set.

HTML

- ▶ HTML stands for HyperText Markup Language, which is read by web browsers to render and display website content.
- ▶ This means that if we want to scrape the content from a particular website, we are often given the HTML code.
- ▶ For us to be able to scrape websites, we must know what HTMLs are and how we can navigate them to retrieve the information we want.

HTML EXAMPLE

- ▶ The elements contained within the angle brackets are called **HTML tags**.
- ▶ Usually come in pairs.
- ▶ The stopping tag contains a forward slash, while the starting does not.

```
<html>
  <body>
    <div>
      <p>Hello!</p>
      <p>I am from ITAM!</p>
    </div>
    <p>Have a nice day! </p>
  </body>
</html>
```

HTML EXAMPLE

- ▶ The tags are nested within others (that's why we usually refer to a **HTML Tree**), which give rise to a hierarchy
- ▶ If we move from left to right we refer to it as moving forward generations.
- ▶ Top to bottom, moving between the same generation, but between siblings.

```
<html>
  <body>
    <div>
      <p>Hello!</p>
      <p>I am from ITAM!</p>
    </div>
    <p>Have a nice day! </p>
  </body>
</html>
```

HTML TAGS AND ATTRIBUTES

- ▶ We have already seen three tag names: the `html`, `div`, and `p` (paragraph) tags. There are many [more](#).
- ▶ The HTML document itself begins with `<html>` and ends with `</html>`.
- ▶ The visible part of the HTML document is between `<body>` and `</body>`.
- ▶ The `<div>` element is used as a container for other HTML elements.
- ▶ Inside each tag, we might encounter **attributes** that define them.

HTML TAGS AND ATTRIBUTES

- ▶ Let's consider a div tag with two attributes: id and class.
- ▶ The id attribute can be used as a unique identifier for the tag element.
- ▶ The class attribute "class1" can also help us identify this div element, but might not be unique.
- ▶ A tag can belong to different classes, which need to be separated by a space.
- ▶ In the following example, the tag belongs to classes *class1* and *class2*

```
<div id="unique-id" class="class1 class2">  
    ..div elements  
</div>
```

NACIONAL

Sube percepción de inseguridad en zonas de CDMX y Cuernavaca

03 MIN 30 SEG

Benito Jiménez

Cd. de México (18 enero 2024) - 10:06 hrs



```
▼<div class="gr_titulo_del_articulo" id="tituloArt" style="grid-column:
1; grid-row: 2;display:block;">
  <h1>Sube percepción de inseguridad en zonas de CDMX y Cuernavaca
  </h1> == $0
</div>
```

Encuesta Nacional de Victimización y Percepción sobre Seguridad Pública (ENVIPE) 2023



ENVIPE

La Encuesta Nacional de Victimización y Percepción sobre Seguridad Pública (ENVIPE) 2023 es la décima tercera entrega de la serie estadística generada por el Subsistema Nacional de Información de Gobierno, Seguridad Pública e Impartición de Justicia (SNIGSPIJ), coordinada por el Instituto Nacional de Estadística y Geografía (INEGI). Con este proyecto se da continuidad a las temáticas abordadas en las ediciones anteriores de esta encuesta, cuyos resultados han sido declarados Información de Interés Nacional por la Junta de Gobierno del INEGI.

[Ver más presentación](#)

Temas relacionados

🕒 Última actualización: [11 de septiembre de 2023](#)

[Documentación](#) [Tabulados](#) [Microdatos](#) **[Datos abiertos](#)** [Herramientas](#)

Buscar en título

Buscar

[Descargar todos los archivos](#)

Resultados encontrados: 1

Estándar de datos abiertos

```
> <a href="javascript:void(0);" id="aDescargar_DMD
A" role="button" aria-expanded="false" tabindex=
"0" aria-label="Descargar todos los archivos en
una carpeta zip" onclick="AbrirDescargaDinamica
('DMDA')" onkeyup="AbrirDescargaDinamica_event
('DMDA',event)">...</a> == $0
```

XPATH

- ▶ We now understand the basic structure of HTMLs, but how can we translate this to something that our computer can digest?
- ▶ XPath notation is the way to go:

```
xpath = '/html/body/div[2]'
```

- ▶ Same syntaxis as folder navigation: The single forward-slash moves us forward one generation.
- ▶ Brackets are used to help specify which element or elements we want to direct to.

XPATH

- Recall the first example of an HTML, suppose we would like to extract the element 'I am from ITAM'.

```
<html>
  <body>
    <div>
      <p>Hello!</p>
      <p>I am from ITAM!</p>
    </div>
    <p>Have a nice day! </p>
  </body>
</html>
```

- The XPath for this would be:

```
xpath = '/html/body/div/p[2]'
```

XPATH

- ▶ If we wanted to extract all div elements from this HTML, we would apply double slashes at the beginning:

```
xpath = '//div'
```

- ▶ Or all the p tags contained inside the first div:

```
xpath = '/html/body/div//p'
```

```
<html>
  <body>
    <div>
      <p>Hello!</p>
      <p>I am from ITAM!</p>
    </div>
    <p>Have a nice day! </p>
  </body>
</html>
```

XPATH: (AT)TRIBUTES

- ▶ Most often than not we would not refer to the tags by their position using brackets, instead we will use the previously defined attributes
- ▶ Let us use a more complicated HTML example, to explore this:

```
<html>
  <body>
    <div id="uid">
      <p class="class-1">Hello!</p>
      <p class="class-2">I am from ITAM!</p>
    </div>
    <p class="class-1 class-2">Have a nice day!
  </p>
</body>
</html>
```

XPATH: (AT)TRIBUTES

- We can use the following syntaxis to extract elements:

```
xpath = '///p[@class="class-1"]' # all tags p, reduced to class='class-1'
xpath = '//*[@id="uid"]' # all elements, reduced to those with
# id tag 'uid'
xpath = '///div[@id="uid"]/p[2]' # all div tags with id='uid',
# reduced to the second p element.
xpath = '//*[contains(@class, "class-1")]' # all tags that contain
# sub-string 'class-1'
```

```
<html>
  <body>
    <div id="uid">
      <p class="class-1">Hello!</p>
      <p class="class-2">I am from ITAM!</p>
    </div>
    <p class="class-1 class-2">Have a nice day!
  </p>
</body>
</html>
```


HTML CODE

- ▶ To properly select the correct XPath's, and extract the desired elements, we must have a sense of how the HTML of the page looks like.
- ▶ To inspect a certain HTML, we must access the webpage, and go to the DevTools (Developer Tools).
- ▶ Or, for a less interactive version of this, by right clicking within the page (or Control+Click on a Mac) and selecting "View Page Source".
- ▶ Here is an example in my Google Chrome:

HTML: SOURCE CODE

The screenshot shows a web browser window displaying the ITAM website. The address bar shows the URL: `eventos.itam.mx/es/evento/seminario-de-perspectivas-economicas-2024`. The website has a green header with the ITAM logo and navigation links: "Eventos y Noticias", "Da de alta un evento", "Suscríbete al boletín", and "Pró".

The main content area features a section titled "Seminario de Perspectivas Económicas 2024" with the following details:

- 12 de enero de 2024
- De 8.00 a 15.30 h
- Auditorio Raúl Baillères, Río Hondo

Below this, there is a text block: "Te invitamos a nuestro tradicional Seminario de Perspectivas Económicas 2024." followed by a link "Regístrate aquí." and a "Programa" section.

The "Programa" section lists:

- 8.00 h - Bienvenida
- Humberto López, Asociación de Ex Alumnos del ITAM, Presidente
- 8.15 h - Inauguración. Palabras del Rector del Instituto Tecnológico Autónomo de México
- Arturo Fernández, ITAM, Rector

A sidebar menu is open on the right, showing a list of categories: "Con", "Ex A", "Cult", "Pos", "Bols", "Vinc", "Dep", "Aca", "Des", "Con", "Exte", "Otro", "Mut".

Overlaid on the right side of the browser window is a menu with various options and keyboard shortcuts:

- Nueva pestaña (Ctrl + T)
- Nueva ventana (Ctrl + N)
- Nueva ventana de incógnito (Ctrl + Mayús + N)
- Eduardo (Persona 1)
- Contraseñas y Autocompletar
- Historial
- Descargas (Ctrl + J)
- Marcadores y listas
- Extensiones
- Borrar datos de navegación... (Ctrl + Mayús + Supr)
- Zoom (100 %)
- Imprimir... (Ctrl + P)
- Buscar esta página en Google...
- Traducir...
- Buscar y editar
- Guardar y compartir
- Más herramientas
- Ayuda
- Configuración

At the bottom of the browser window, a yellow bar displays: "Herramientas para desarrolladores (Ctrl + Mayús + I)".

INSPECTING THE HTML

- ▶ Suppose we would like to extract the date of these events, since they started.
- ▶ We can use another tool all browsers have: 'Inspect Element'.
- ▶ This will direct you to the actual HTML Code for that specific element.
- ▶ For this, let your mouse hover over the element you're interested in, then right-click (or Control+Click on a Mac), and select 'Inspect Element'.
- ▶ The following panel will be opened:

INSPECTING THE HTML

- ▶ After quickly inspecting it, using what we have learned, we arrive to a unique id *fecha-evento*.
- ▶ We can set up our XPath that would theoretically lead us there:

```
xpath = '//div[@id="fecha-evento"]'
```

- ▶ Now, how can we translate this to code?

WEB SCRAPING IN PYTHON

- ▶ Python is a popular language for web scraping due to its rich ecosystem of libraries and tools.
- ▶ Two of the most commonly used are `selenium` and `scrapy`. We'll go over both during this tutorial.
- ▶ We'll start with `scrapy`. Let's install the necessary packages:

```
!pip install requests  
!pip install scrapy
```

REQUESTS

- ▶ You might be wondering why we installed the package `requests` without mentioning it.
- ▶ This package allows us to obtain the full HTML of the webpage we would like to extract, in a non-dynamic way (no clicking, entering a page, etc.)
- ▶ It is a great starting point for web scraping:

```
import requests
url = 'https://eventos.itam.mx/es/evento/seminario-de-perspectivas-economicas-2024'
html = requests.get(url).content
```

SCRAPY SELECTOR

- The output is a bit messy, however, we can clean it up using regex:

```
import re

def clean_string(input_string):
    # Remove everything inside '<>'
    cleaned_string = re.sub(r'<[^>]+>', ' ', input_string)

    # Remove '\'' and the letter right next to it
    cleaned_string = re.sub(r'\\.', '', cleaned_string)

    # Remove any double spaces
    cleaned_string = re.sub(r'\\s+', ' ', cleaned_string)

    return cleaned_string

fecha_clean = clean_string(fecha[0])
fecha_clean
##### '12 de enero de 2024 De 8.00 a 15.30 h'
```

SCRAPING ITAM'S WEBPAGE

- ▶ We might be interested in scraping automatically all the speakers from ITAM's [Seminario de Perspectivas Económicas](#), across the years.
- ▶ We can start by scraping the ones from 2024 and later expand it with a loop to all available years.
- ▶ After quickly inspecting the webpage, we notice we just need the elements with `id = 'cuerpo-evento'` that are not in the class `'rtecenter'`:

```
evento = sel.xpath('//div[@id="cuerpo-evento"]//p[@dir="ltr" and  
'not(@class="rtecenter")']).extract()
```

SCRAPING ITAM'S WEBPAGE

- We clean it up using the previously defined function and obtain the following:

```
evento_clean = []
for ev in evento:
    event = clean_string(ev)
    evento_clean.append(event)
evento_clean

#####: [..., '8.00 h Bienvenida',
##### 'Humberto López, Asociación de Ex Alumnos
##### del ITAM, Presidente',
##### '8.15 h Inauguración. Palabras del
#####Rector del Instituto Tecnológico', ...]
```

- Notice we can extract the names, if that particular string contains commas.

SCRAPING ITAM's WEBPAGE

- Using that, we can then turn into a data frame and extract the name, occupation and firm they work for.

```
import pandas as pd

people = [element for element in evento_clean if ',' in element]
people_df = pd.DataFrame(people, columns=['name'])
people_df['speaker'] = people_df['name'].apply(lambda x: x.split(',')[0].strip())
people_df['moderator'] = people_df['speaker'].apply(lambda x: 1 if 'Moderador:' in x
                                                    or 'Moderadora:' in x else 0)
people_df['speaker'] = people_df['speaker'].str.replace('Moderador\\:|Moderadora\\:',
                                                         '', regex=True).str.strip()
people_df['occupation'] = people_df['name'].str.extract(r',(.*)')
people_df['year'] = 2024
people_df = people_df.drop(['name'], axis = 1)
people_df
```

- Finally, we obtain a clean DataFrame:

	speaker	moderator	occupation	year
0	Humberto López	0	Asociación de Ex Alumnos del ITAM, Presidente	2024
1	Arturo Fernández	0	ITAM, Rector	2024
2	Ilan Goldfajn	0	Banco Interamericano de Desarrollo, Presidente	2024
3	Carlos Capistrán	0	Bank of America, Economista en Jefe para Méxi...	2024
4	Ernesto Revilla	0	Citigroup, Economista en Jefe para América La...	2024
5	Alejandrina Salcedo	0	Banco de México, Directora General de Investi...	2024
6	Miguel Messmacher	1	ITAM, Director general de la División Académi...	2024
7	Luis Carlos Ugalde	0	Integralia Consultores, Director General	2024
8	Federico Reyes-Heróles	0	Transparencia Mexicana, Presidente del Consej...	2024
9	Agustín Basave	0	Universidad de Monterrey, Director del Instit...	2024
10	Alexandra Uribe	1	ITAM , Directora de la Licenciatura en Cienci...	2024
11	Alan Stoga	0	Tallberg Foundation, Presidente	2024
12	Natalia Saltalamacchia	1	ITAM, Jefa del Departamento Académico de Estu...	2024
13	Eduardo Garza T.	0	Friza , Presidente y Fundador	2024
14	Hans-Joachim Kohlsdorf	0	Energy to Market , Socio Fundador	2024
15	Marlene Garayzar	0	Stori , Cofundadora	2024
16	Guillermo I. García Alcocer	1	ITAM, Director de Planeación y Evaluación de ...	2024
17	Carlos Slim	0	Fundación Telmex, Presidente	2024
18	Rafael Fernández de Castro	0	Centro de Estudios México-Estados Unidos UCSD...	2024
19	Arturo Sarukhán	0	Sarukhán y asociados, Presidente	2024
20	Jorge Suárez-Vélez	0	Allen & Company, Director	2024
21	Ana María Salazar	1	Grupo Salazar Slack SC, Directora	2024
22	Alejandro Hernández	0	ITAM, Vicerrector	2024

- ▶ We can easily do it for more years, let's say, 2020 to 2024.
- ▶ Let us just define a more general function and run a loop through the url:

```
def scrape_seminario(year):
    url = f'https://eventos.itam.mx/es/evento/seminario-de-perspectivas-economicas-{year}'
    html = requests.get(url).content
    sel = Selector(text = html)
    evento = sel.xpath('//div[@id="cuerpo-evento"]//p').extract()

    evento_clean = []
    for ev in evento:
        event = clean_string(ev)
        evento_clean.append(event)

    people = [element for element in evento_clean if ',' in element]
    people_df = pd.DataFrame(people, columns=['name'])
    people_df = people_df[~people_df['name'].str.contains('Cuota de recuperación\\\:|reembolso')]
    people_df['speaker'] = people_df['name'].apply(lambda x: x.split(',')[0].strip())
    people_df['moderator'] = people_df['speaker'].apply(lambda x: 1 if 'Moderador:' in x
                                                         or 'Moderadora:' in x or 'Modera:' in x else 0)
    people_df['speaker'] = people_df['speaker'].str.replace('Moderador\\\:|Moderadora\\\:|Modera\\\:|',
                                                            '', regex=True).str.strip()

    people_df['occupation'] = people_df['name'].str.extract(r',(.*)')
    people_df['year'] = year
    people_df = people_df.drop(['name'], axis = 1)

    return(people_df)
```

► We run the loop:

```
df_final = pd.DataFrame()
for y in range(2020, 2025):
    df = scrape_seminario(y)
    df_final = pd.concat([df_final, df]).reset_index(drop=True)

df_final
```

► And we get the following DB:

	speaker	moderator	occupation	year
0	Arturo Fernández P.	0	ITAM , Rector	2020
1	Alejandro Díaz de León Carrillo	0	Banco de México, Gobernador	2020
2	Alejandro Werner Wainfeld	0	FMI, Dir. Depto. del Hemisferio Occidental	2020
3	Iván Moguel	0	Chévez Ruiz Zamarripa, Socio	2020
4	Lorenza Martinez	1	Accenture México, Managing Director	2020
...	...	...	...	...
86	Rafael Fernández de Castro	0	Centro de Estudios México-Estados Unidos UCSD...	2024
87	Arturo Sarukhán	0	Sarukhán y asociados, Presidente	2024
88	Jorge Suárez-Vélez	0	Allen & Company, Director	2024
89	Ana María Salazar	1	Grupo Salazar Slack SC, Directora	2024
90	Alejandro Hernández	0	ITAM, Vicerrector	2024

91 rows × 4 columns

EXTRA: CSS LOCATORS

- ▶ CSS Locators is a another way to identify elements in an HTML.
- ▶ CSS stands for Cascading Style Sheets, which describes how the elements are displayed on the screen.
- ▶ Basically the same as XPath, just the notation changes:
- ▶ '/' changes to '>', '// ' to ' ' and '[]' to ':nth-of-type()'

XPath:

```
xpath = '/html/body//div/p[2]'
```

CSS

```
css = 'html > body div > p:nth-of-type(2)'
```

EXTRA: CSS LOCATORS

- ▶ One good reason to use CSS Locators instead of XPath is how simple it is to identify attributes:
- ▶ To find an element by class, use '.': p.class1
- ▶ To find an element by id, use '#': p#uid

p elements with class-1 inside div with uid

css_selector = 'div#uid > p.class1'

all elements whose class belongs to class-1

css_selector = '.class1'

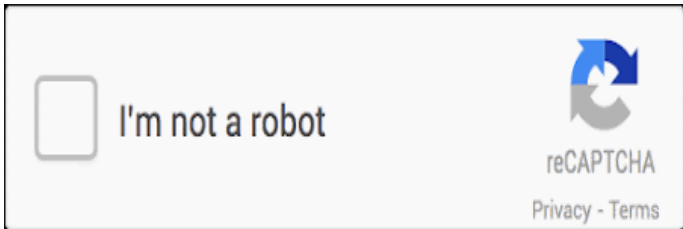
Selector is the same, with css instead:

sel = Selector(text = html)

sel.css(css_selector)

BUILDING A BOT

- ▶ Up until now we have focused on a non-dynamic way of scraping.
- ▶ Meaning we have not interacted with web-pages as we, as humans, would normally do.
- ▶ For example, a lot of content is often protected by a username and a password, or we must scroll through the web-page to find it.



BUILDING A BOT WITH SELENIUM

- ▶ Selenium package is by definition a test automation tool that has a webdriver component with scraping tools.
- ▶ By test automation we refer to a bot, that interacts with webpages as we humans would.
- ▶ Bots are commonly used to like, RT and comment tons of social media posts, send out emails automatically, etc.
- ▶ We can leverage this functionality, and what we have learnt from XPaths and CSS, to enter different webpages and extract information from them.

SELENIUM

- ▶ Let's set up the webdriver:¹

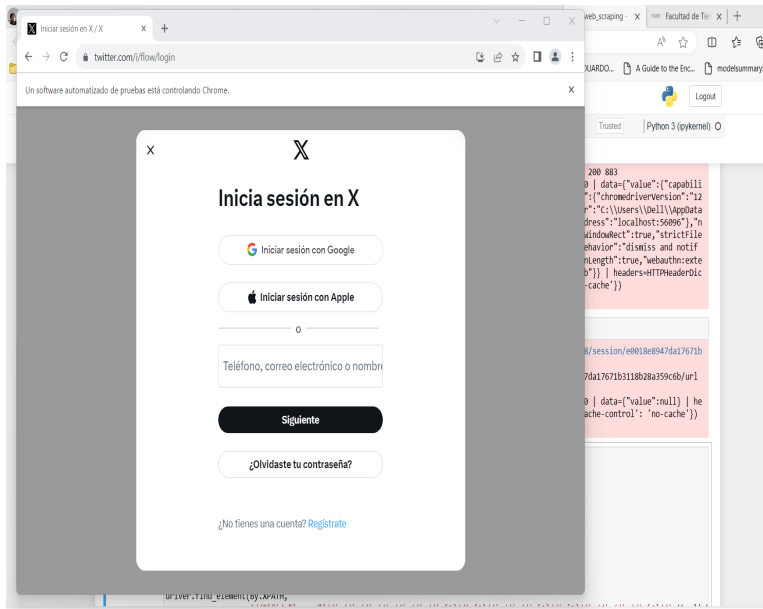
```
!pip install selenium
from selenium import webdriver

# Open Chrome
driver = webdriver.Chrome()

# Open the desired URL:
driver.get('http://twitter.com/login')
```

- ▶ Notice a Chrome window appears automatically:

¹The Jupyter Notebook contains more instructions if you have any trouble.



SELENIUM FUNCTIONS

- We can interact with this window by locating their XPath's, and using the functions `send_keys()` and `click()`.

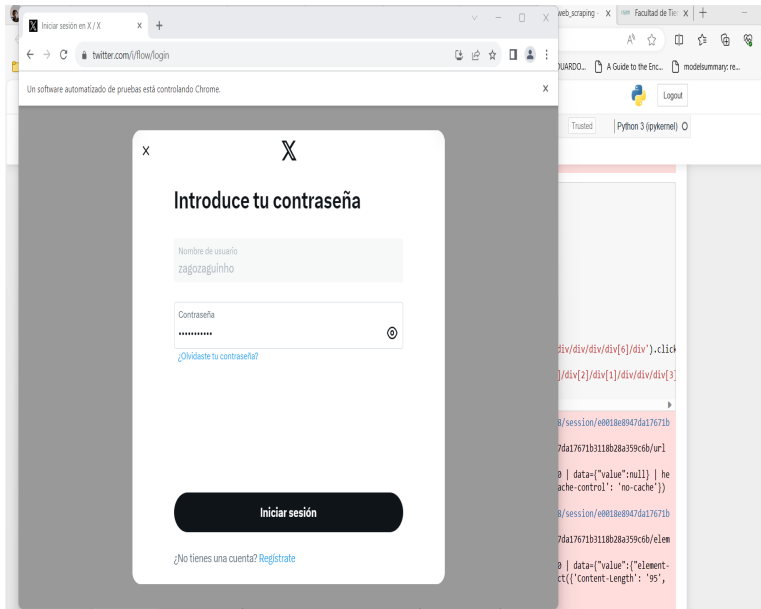
```
driver.find_element(By.NAME, 'text').send_keys(handle)

time.sleep(3)
driver.find_element(By.XPATH, '//xpath_SIGUIENTE').click()

time.sleep(3)
driver.find_element(By.XPATH, '//*xpath_PASSWORD').send_keys(password)

time.sleep(7)
driver.find_element(By.XPATH, '//xpath_ENTER_TWITTER').click()
```

- Notice this is dynamic, we must wait a few seconds so that the page loads, and Twitter does not suspect we are bots.



SCRAPING TWITTER

- ▶ Up until now we have just used the automatization techniques that Selenium offers us.
- ▶ To extract meaningful information, let's say, the date where a certain profile entered Twitter, we must use the function `text` and the corresponding XPath.

```
driver.get(f'https://twitter.com/ZagoZaguinho')

profile = driver.find_elements(By.XPATH,
                               '//span[@data-testid="UserJoinDate"]') # Element List

for p in profile:
    Date = p.text

Date
### 'Joined March 2023'
```

SCRAPING TWITTER

- We can loop to get this information from a lot of accounts.

```
for acc in ['ZagoZaguinho', 'HLarreguy']:
    driver.get(f'https://twitter.com/{acc}')

    profile = driver.find_elements(By.XPATH,
        '//span[@data-testid="UserJoinDate"]') # Element List
    for p in profile:
        Date = p.text

    print(Date)
    time.sleep(10)
    ### 'Joined March 2023'
    ### 'Joined February 2021'
```

SCRAPING TWITTER

- We can also scrape some other useful things, such as the Trending Topics at the moment:

```
driver.get(f'https://twitter.com/explore/tabs/trending')
time.sleep(5)

trend = driver.find_elements(By.XPATH,
    '//*[@data-testid="Some_Long_ID" and not(@style="Some_Long_Style")]/div/div')

Name = []
posts = []
for t in trend:
    Name1 = t.find_element(By.XPATH, '//*[@style="some_long_style"]').text
    Name.append(Name1)

    posts1 = t.find_element(By.XPATH, './div[3]').text
    posts.append(posts1)

df = pd.DataFrame(zip(Name, posts),
    columns=['trend', 'n_posts'])
```

	trend	n_posts
0	#EquipoADMM	2,000 posts
1	Epstein	2.3M posts
2	Costco	18.4K posts
3	Reyes	182K posts
4	#QUACKITYPLEASE	24.6K posts
5	#ViernesDeBonosPlaydoit	
6	Chicharito	18.2K posts
7	SCJN	83.2K posts
8	Lenia Batres	42.5K posts
9	Stephen Hawking	215K posts
10	Constitución	57.6K posts
11	Michael Jackson	178K posts
12	Corte	115K posts
13	Chivas	29.4K posts
14	Ernestina Godoy	14.6K posts
15	León Larregui	6,584 posts
16	David Soul	28.7K posts
17	Cowell	7,474 posts
18	Skyler	3,795 posts
19	Situación de Calle	2,131 posts
20	Oprah	81.9K posts

SCRAPING TWITTER

- ▶ Finally, we could also want to scrape the Tweets and ReTweets from a certain user in a certain period of time.
- ▶ To do this, we would have to scroll down through that user's page to visualize correctly each Tweet.
- ▶ We can scroll down a page using Selenium's function: `execute_script()`

```
scroll_distance = 3000 # For Twitter this works well
```

```
driver.execute_script(f'window.scrollTo(0, {scroll_distance});')
```

SCRAPING TWITTER

- ▶ To properly scroll down a webpage we must use a `while` statement, that breaks out of the loop when we've reached the end of the page.
- ▶ Let's look at a simple example:

```
driver.get(f'https://twitter.com/ZagoZaguinho1/with_replies')
scroll_distance = 2500
scroll_pos=[1]
i = 0
t = 0
while True:
    time.sleep(2)
    current_scroll_pos = driver.execute_script("return window.scrollY;")
    scroll_pos.append(current_scroll_pos)

    driver.execute_script(f'window.scrollTo(0, {scroll_distance});')
    time.sleep(5)
    t+=1
    print('Scroll ', i)
    print(scroll_pos[t-1], scroll_pos[t])
    if (scroll_pos[t-1] == scroll_pos[t]):
        print("you've reached the end")
        break

    i+=1
```

SCRAPING TWITTER

- ▶ The last chunk of code just scrolls down through a user profile, but how can we scrape the Tweets and RTs from that user?
- ▶ Using XPaths, at each stage of the loop.
- ▶ To do this, we must extract the relevant HTML information at each scroll:

```
# Initialize the lists
UserTags=[]
TimeStamps=[]
Tweets=[]
scroll_pos=[1]
old_tweet='new'

# 1st scroll HTML:
articles = driver.find_elements(By.XPATH,"//article[@data-testid='tweet']")
```

SCRAPING TWITTER

```
t=0
while True:
    time.sleep(2)
    current_scroll_pos = driver.execute_script("return window.scrollY;")
    scroll_pos.append(current_scroll_pos)

    for article in articles: # for to extract the text from the HTML

        try: # try-except, so that if we do not find anything, we do not get an error.
            UserTag = article.find_element(By.XPATH, "//*[@data-testid='User-Name']").text
            UserTags.append(UserTag)
        except:
            UserTags.append(np.nan)
        try: # Get time of the tweet
            TimeStamp = article.find_element(By.XPATH, "//*[@time]").get_attribute('datetime')
            TimeStamps.append(TimeStamp)
        except:
            TimeStamps.append(np.nan)

        try: # get Tweet text
            Tweet = article.find_element(By.XPATH, "//*[@data-testid='tweetText']").text
            Tweets.append(Tweet)
        except:
            Tweets.append(np.nan)

## continues...
```

SCRAPING TWITTER

```
while True:
    ....
    ....
    # scroll down
    driver.execute_script(f'window.scrollBy(0, {scroll_distance});')
    time.sleep(5)
    # extract elements from next scroll
    articles = driver.find_elements(By.XPATH, "//article[@data-testid='tweet']")
    t+=1

    print(scroll_pos[t-1], scroll_pos[t])
    # Condition to end the loop:
    if (scroll_pos[t-1] == scroll_pos[t]):
        print("you've reached the end")
        break

df = pd.DataFrame(zip(UserTags, TimeStamps, Tweets),
                  columns=['username', 'TimeStamp', 'text'])
```

SCRAPING TWITTER

- ▶ This code goes to the last Tweet available, either because Twitter isn't showing any more or because it is the last one.
- ▶ We might want to stop at a certain point in time.
- ▶ To do that, we must need to add a condition on the TimeStamp we are already scraping:

```
if ((datetime.fromisoformat(TimeStamp[-1]) < reference_date)):
    old_tweet = 'old'
    print("you've reached a tweet older than the reference tweet")
...
...
...

if (scroll_pos[t-1] == scroll_pos[t]) or (old_tweet == 'old'):
    print("you've reached the end")
    break
```

	username	TimeStamp	text
0	Zago\n@ZagoZaguinho1\n\n17h	2024-01-18T06:55:59.000Z	También tu última comparación es engañosa, com...
1	Xóchitl Gálvez Ruiz\n@XochitlGalvez\n\nJan 17	2024-01-17T18:04:42.000Z	Por su boca muere el pez.\n\n#MéxicoVaConX
2	Zago\n@ZagoZaguinho1\n\nJan 17	2024-01-17T21:26:17.000Z	Hahahahaha artell!
3	Arturo Ángel\n@arturoangel20\n\nJan 17	2024-01-17T17:09:45.000Z	Este gráfico está tan mal en tantos aspectos m...
4	Zago\n@ZagoZaguinho1\n\nJan 17	2024-01-17T18:51:14.000Z	A que no puedes nombrar ni uno
...	...	...	...
142	Zago\n@ZagoZaguinho1\n\nDec 22, 2023	2023-12-22T20:08:57.000Z	Aunque algo ha de haber ayudado! Parece ser qu...
143	Emilio Gutierrez\n@emiliogf\n\nDec 22, 2023	2023-12-22T15:50:01.000Z	Feliz de que ya está disponible en línea este ...
144	Zago\n@ZagoZaguinho1\n\nDec 22, 2023	2023-12-22T19:29:42.000Z	Muchas felicidades!
145	NBA\n@NBA\n\nDec 21, 2023	2023-12-22T04:39:16.000Z	"I don't really know where that came from." \n...
146	Zago\n@ZagoZaguinho1\n\nDec 20, 2023	2023-12-20T20:32:10.000Z	

147 rows × 3 columns

SCRAPING NEWSPAPER SITES

- ▶ Finally, using these scraping techniques we can also identify major events in newspapers.
- ▶ Obviously, this would take a lot of difficult object oriented programming, and a more profound understanding of web-crawlers, but we have some code that we can use.
- ▶ [Jose Ramón Enriquez](#) built a code that allows us to look for keywords in over 500 Mexican newspapers.
- ▶ We can also use this code to identify if some events happened at a certain municipality, at a certain point in time.

SCRAPING NEWSPAPER SITES

- ▶ Another package we can use to scrape websites is BeautifulSoup.
- ▶ This contains the same functions as we have seen for scrapy and selenium, but the syntax is a little bit friendlier.
- ▶ Let us look at a chunk of Jose Ramón code as an example.
- ▶ First, we must open the url, extract the HTML and parse it.

```
response = ul.urlopen(url) ## Selector List
wc        = response.read()

# parse webpage
soup = BeautifulSoup(wc, 'html.parser') # Like scrapy
```

SCRAPING NEWSPAPER SITES

- ▶ Then, we can use the functions such as `find_all()`.
- ▶ We must find all HTML elements with a specific tag name (`heads['name']`) and specific attributes (`heads['attrs']`) within the parsed HTML document (`soup`).
- ▶ The resulting elements are returned as a `ResultSet`.

```
articles = soup.find_all(heads['name'], attrs=heads['attrs']) # Same as using XPATHs
for art in articles:
    title = art.find(titles['name'], attrs=titles['attrs'])
    title = art
    art_url = title.find('a').get('href')
    summary = art.find(dates['name'], attrs=dates['attrs'])
```

```
* http://abc.miled.com/  
$ header class="entry-header"  
^ h2 class="entry-title", a href  
@ span class="auth-posted-on", time datetime  
* http://www.campechehoy.mx/  
$ div class="td-block-span6"  
^ h3 class="entry-title td-module-title", a href  
@ span class="td-post-date", time datetime
```

SCRAPING NEWSPAPER SITES

- And here is the complete function. This function allows you to scrape the content (title, name, date and url) of a newspaper article.

```
def wp4(url, struct):
    # fetch url
    try:
        response = ul.urlopen(url) ## Selector List
        wc       = response.read()
    except Exception as e:
        raise Exception(e)

    # parse webpage
    soup = BeautifulSoup(wc, 'html.parser') # Like scrapy
    result = []
    if heads:
        articles = soup.find_all(heads['name'], attrs=heads['attrs']) # Same as using XPATHs
        for art in articles:
            if titles['name']:
                title = art.find(titles['name'], attrs=titles['attrs'])
            else:
                title = art
            art_url = title.find('a').get('href')
            summary = art.find(dates['name'], attrs=dates['attrs'])
            date = get_date(summary, dates['extra'])
            result.append({'url': art_url, 'title': title, 'date': date})
```

SCRAPING NEWSPAPER SITES

- ▶ As you can see, the code is very complicated, so it is overkill to go over it in this lecture. However, we can still use it for our own purposes.
- ▶ The code, fortunately, generalizes well to different keywords, therefore, we just need to change those inputs.
- ▶ Keywords are inputted as the following:

"marcha AND feminista AND Aguascalientes"
"marcha AND mujeres AND Aguascalientes"
"marcha AND feminista AND Asientos"
"marcha AND mujeres AND Asientos"
"marcha AND feminista AND Calvillo"
"marcha AND mujeres AND Calvillo"
"marcha AND feminista AND Cosío"
"marcha AND mujeres AND Cosío"
"marcha AND feminista AND Jesús María"
"marcha AND mujeres AND Jesús María"
"marcha AND feminista AND Pabellón de Arteaga"
"marcha AND mujeres AND Pabellón de Arteaga"
"marcha AND feminista AND Rincón de Romos"
"marcha AND mujeres AND Rincón de Romos"
"marcha AND feminista AND San José de Gracia"
"marcha AND mujeres AND San José de Gracia"
"marcha AND feminista AND Tepezalá"
"marcha AND mujeres AND Tepezalá"
"marcha AND feminista AND El Llano"
"marcha AND mujeres AND El Llano"
"marcha AND feminista AND San Francisco de los Romo"
"marcha AND mujeres AND San Francisco de los Romo"
"marcha AND feminista AND Ensenada"
"marcha AND mujeres AND Ensenada"
"marcha AND feminista AND Mexicali"
"marcha AND mujeres AND Mexicali"

SCRAPING NEWSPAPER SITES

- To do this, we just need to concatenate the strings:

```
municipalities_file = '../..data/preprocess/municipalities.txt'
output_file = '../..terms_short_test_marchaAND8m.txt'

with open(municipalities_file, 'r') as municipalities_f, open(output_file, 'w') as output_f:
    municipalities = municipalities_f.read().splitlines()
    search_terms = ['marcha', 'feminista'] # Add your search terms here

    for municipality in municipalities:
        combined_terms = ' AND '.join([f'{term} AND {municipality}' for
                                         term in search_terms])
        output_f.write(combined_terms + '\n')
```

SCRAPING NEWSPAPER SITES

- ▶ The code outputs the name of the article, the title, as well as the URL and the date of creation.
- ▶ From the title one can infer if these are relevant events, or errors.
- ▶ We could also use the techniques we have seen to scrape the content inside each URL, using a webcrawler.

marcha AND http://www.hidrocalidodigital.com/todos-a-guardar-sana-distancia/	Todos a guardar sana distancia	2020-03-23T04:02:30
marcha AND http://www.hidrocalidodigital.com/estiman-que-no-abriran-mas-de-la-mitad-de-escuelas	Estiman que no abrirán más de la mitad de escuelas	2020-03-09T16:42:42
marcha AND http://www.hidrocalidodigital.com/miles-de-mujeres-hidrocalidas-salieron	Miles de mujeres hidrocalidas salieron a las calles para manifestarse en contra de la violencia de género	2020-03-09T00:51:01
marcha AND http://www.hidrocalidodigital.com/para-dona-albinita-no-hay-impedimento	Para Doña Albinita no hay impedimento para salir adelante	2020-03-08T19:14:12
marcha AND https://impulsoedomex.com.mx/?p=223564	Sororidad de Mujeres en Toluca se unen en una sola Voz	2020-03-09T04:19:40
marcha AND https://lucedselsiglo.com/?p=70496	Protestan mujeres en estados	2020-03-08T15:10:39
marcha AND https://lucedselsiglo.com/?p=72730	Sanitizan con cuidado terminales portuarias	2020-03-25T06:28:15
marcha AND https://lucedselsiglo.com/?p=72471	Cancún 50 años A imagen y semejanza de Cancún	2020-03-23T07:25:57
marcha AND https://lucedselsiglo.com/?p=70496	Protestan mujeres en estados	2020-03-08T15:10:39
marcha AND https://www.rotativooaxaca.com.mx/?p=81789	En Huajuapán participan trabajadoras del volante en marcha feminista	2020-03-10T06:10:28
marcha AND http://slurpie.laurag.tv/2020/03/no-sabiamos-lo-del-paro-del-lunes-9-la-verdad-no-me-creyeron/#8221; afirma AMLO	“No sabíamos lo del paro del lunes 9 la verdad no me creyeron” afirma AMLO	2020-03-06T08:07:22
marcha AND http://slurpie.laurag.tv/2020/03/no-sabiamos-lo-del-paro-del-lunes-9-la-verdad-no-me-creyeron/#8221; afirma AMLO	“No sabíamos lo del paro del lunes 9 la verdad no me creyeron” afirma AMLO	2020-03-06T08:07:22
marcha AND https://abc.milled.com/?p=7697	Convocan a marchas en todo el país por Día de la Mujer	2020-03-07T20:53:03
marcha AND https://amqueretaro.com/?p=774044	Alistan las marchas por el Día de la Mujer en todo el país	2020-03-07T19:01:37
marcha AND http://despertardelacosta.net/?p=29651	Estado de los ESTADOS	2020-03-10T11:55:53
marcha AND https://elrbe.com/?p=117551	Protestan Mujeres en México por Alto Nivel de Inseguridad y Feminicidios	2020-03-09T08:35:28
marcha AND https://heraldodepuebla.com/?p=39684	En tiempo real	2020-03-10T11:24:56
marcha AND https://impulsoedomex.com.mx/?p=223564	Sororidad de Mujeres en Toluca se unen en una sola Voz	2020-03-09T04:19:40
marcha AND https://www.lavozdemichoacan.com.mx/?p=734712	Morelia se viste de morado al grito de "Ni una menos" inicia la marcha de mujeres	2020-03-08T17:14:50
marcha AND https://www.lavozdemichoacan.com.mx/?p=734712	Morelia se viste de morado al grito de "Ni una menos" inicia la marcha de mujeres	2020-03-08T17:14:50
marcha AND http://suracapulco.mx/?p=298260	Marchan y exigen justicia en Taxco por crimen de joven de 18 años; colocan memorial en Plaza Borda	2020-03-28T05:20:09
marcha AND https://www.elsiglo.com.mx/?p=76477	Protestan mujeres de Toluca por casos de feminicidios y violencia de género	2020-03-09T04:19:40

QUESTIONS